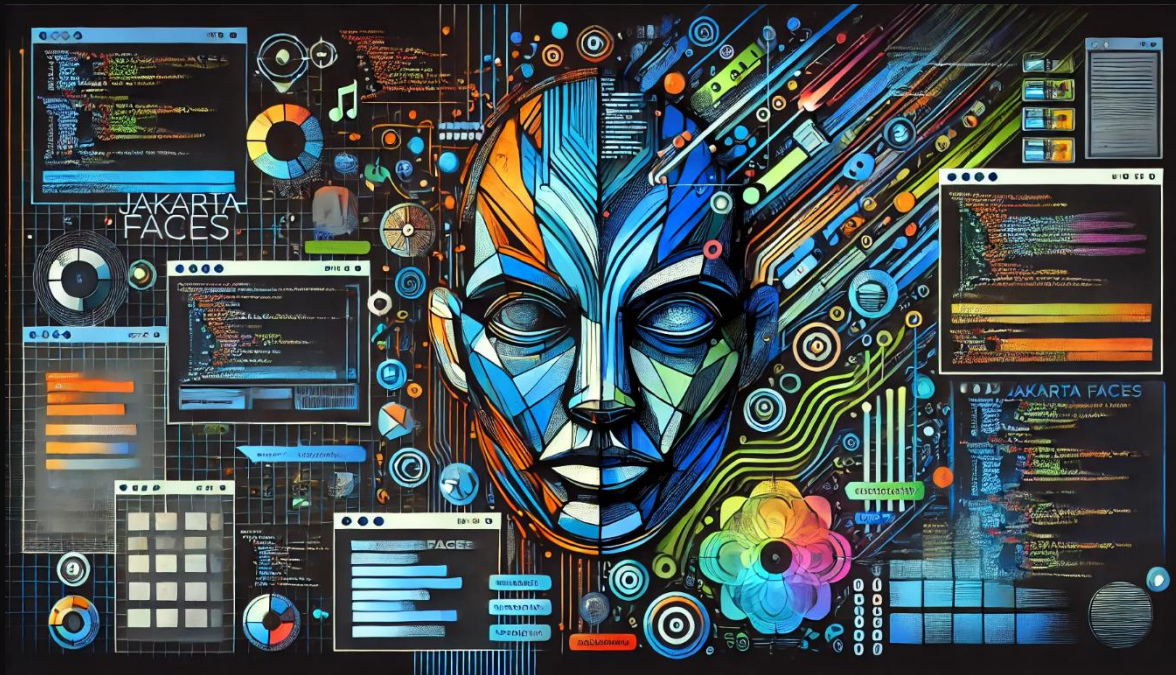


Hola Mundo con Jakarta Faces



En esta lección vamos a crear nuestra primera aplicación de Hola Mundo con JSF.

1. Crear un proyecto Web con Maven

Crea un nuevo proyecto Web con Maven en tu IDE y confirma que se hayan añadido las librerías de jakarta ee en su archivo pom.xml.

El proyecto se llama: HolaMundoJSF

Si creaste tu archivo como un proyecto Web de Maven con Netbeans no tienes que hacer nada más que revisar que todo se haya generado correctamente.

De manera opcional puedes configurar el código fuente del jdk a la versión 21 dentro del archivo pom.xml:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>web</groupId>
  <artifactId>HolaMundoJSF</artifactId>
  <version>1.0</version>
  <packaging>war</packaging>
```

```

<name>HolaMundoJSF</name>

<properties>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  <jakartaee>11.0.0-M1</jakartaee>
</properties>

<dependencies>
  <dependency>
    <groupId>jakarta.platform</groupId>
    <artifactId>jakarta.jakartaee-api</artifactId>
    <version>${jakartaee}</version>
    <scope>provided</scope>
  </dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.12.1</version>
      <configuration>
        <source>21</source>
        <target>21</target>
      </configuration>
    </plugin>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-war-plugin</artifactId>
      <version>3.4.0</version>
    </plugin>
  </plugins>
</build>
</project>

```

Creación de una aplicación básica de "Hola Mundo"

Esta es la estructura que crearemos para nuestro proyecto:

```

HolaMundoJSF
├── pom.xml
├── src
│   └── main
│       └── webapp
│           └── WEB-INF

```

```

├── web.xml (opcional)
├── index.xhtml
└── faces-config.xml (opcional)

```

Nota: Eliminamos el archivo index.html, no lo vamos a necesitar.

Archivo **web.xml**. Este archivo nos permite agregar la configuración básica para que nuestra aplicación de JSF funcione correctamente.

```

<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
https://jakarta.ee/xml/ns/jakartaee/web-app_6_0.xsd"
         version="6.0">
    <servlet>
        <servlet-name>FacesServlet</servlet-name>
        <servlet-class>jakarta.faces.webapp.FacesServlet</servlet-class>
        <load-on-startup>1</load-on-startup>
    </servlet>
    <servlet-mapping>
        <servlet-name>FacesServlet</servlet-name>
        <url-pattern>*.xhtml</url-pattern>
    </servlet-mapping>
    <welcome-file-list>
        <welcome-file>index.xhtml</welcome-file>
    </welcome-file-list>
</web-app>

```

Archivo **index.xhtml**

Creamos el archivo index.xhtml en la carpeta src/main/webapp

```

<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:h="jakarta.faces.html">
<h:head>
    <meta charset="UTF-8"/>
    <title>Hola Mundo con Jakarta Faces</title>
</h:head>
<h:body>
    <h:outputText value="Hola Mundo con JSF" />
    <br/>
    <h:outputLabel value="Saludos!" />
</h:body>

```

```
</html>
```

Explicación de archivo web.xml:

JSF requiere el mapeo del `FacesServlet` para procesar las páginas `.xhtml` y renderizar correctamente la interfaz de usuario.

1. Declaración del `FacesServlet`:

```
<servlet>
  <servlet-name>FacesServlet</servlet-name>
  <servlet-class>jakarta.faces.webapp.FacesServlet</servlet-class>
  <load-on-startup>1</load-on-startup>
</servlet>
```

- **FacesServlet:** Es el controlador central de JSF que procesa las solicitudes y respuestas para las páginas JSF.
- **load-on-startup:** Indica que el servlet se carga al iniciar la aplicación, lo cual es necesario para que JSF funcione.

2. Mapeo del `FacesServlet`:

```
<servlet-mapping>
  <servlet-name>FacesServlet</servlet-name>
  <url-pattern>*.xhtml</url-pattern>
</servlet-mapping>
```

- Esto asocia el `FacesServlet` con las páginas `.xhtml`. Sin este mapeo, el servidor no sabe cómo procesar esas páginas y no mostrará nada.

Página de bienvenida:

```
<welcome-file-list>
  <welcome-file>index.xhtml</welcome-file>
</welcome-file-list>
```

- Define la página inicial de la aplicación cuando se accede al contexto raíz.

Desde las versiones más recientes de **Jakarta EE (anteriormente Java EE)** y **JSF**, algunas configuraciones de `web.xml` se pueden omitir si usas anotaciones. Sin embargo, el mapeo del `FacesServlet` es **crítico** si no defines explícitamente estas configuraciones en tu proyecto.

Alternativa con Anotaciones

Si prefieres no usar `web.xml`, puedes configurar el `FacesServlet` directamente con anotaciones en lugar de usar el archivo `web.xml`:

1. Crea una clase de configuración para registrar el servlet:

```
import jakarta.faces.webapp.FacesServlet;
import jakarta.servlet.ServletContext;
import jakarta.servlet.ServletException;
import jakarta.servlet.ServletRegistration;

import jakarta.servlet.annotation.WebListener;

@WebListener
public class FacesServletInitializer implements jakarta.servlet.ServletContextListener {
    @Override
    public void contextInitialized(ServletContext servletContext) throws ServletException {
        ServletRegistration.Dynamic facesServlet =
            servletContext.addServlet("FacesServlet", FacesServlet.class);
        facesServlet.setLoadOnStartup(1);
        facesServlet.addMapping("*.xhtml");
    }
}
```

2. Define la página de bienvenida en un archivo `META-INF/resources` sin usar `web.xml`.

Los elementos que mencionas (`context-param`) son **opcionales**, pero son altamente recomendados dependiendo del entorno en el que estés trabajando y tus necesidades de configuración. Aquí tienes una explicación sobre lo que hacen y otros elementos que podrías considerar:

1. Parámetros opcionales en el archivo `web.xml`:

a) `jakarta.faces.FACELETS_REFRESH_PERIOD`

```
<context-param>
  <param-name>jakarta.faces.FACELETS_REFRESH_PERIOD</param-name>
  <param-value>0</param-value>
</context-param>
```

- **Función:** Controla cada cuánto tiempo JSF verifica si los archivos `.xhtml` han cambiado para recargarlos.

- **Valor recomendado:**
 - **Desarrollo:** 0 (verifica los cambios en cada solicitud, útil para ver las actualizaciones al instante).
 - **Producción:** -1 (desactiva la verificación para mejorar el rendimiento).

b) `jakarta.faces.PROJECT_STAGE`

```
<context-param>
  <param-name>jakarta.faces.PROJECT_STAGE</param-name>
  <param-value>Development</param-value>
</context-param>
```

- **Función:** Define el estado actual de la aplicación. Puede ser:
 - **Development:** Para entornos de desarrollo. Habilita más logs y errores descriptivos.
 - **Production:** Para entornos de producción. Desactiva características innecesarias y mejora el rendimiento.
- **Valor recomendado:**
 - Usa **Development** en desarrollo y **Production** en producción.

2. Parámetros adicionales recomendados:

a) `jakarta.faces.DEFAULT_SUFFIX`

```
<context-param>
  <param-name>jakarta.faces.DEFAULT_SUFFIX</param-name>
  <param-value>.xhtml</param-value>
</context-param>
```

- **Función:** Define la extensión predeterminada para las páginas JSF. Aunque `.xhtml` es el estándar, puedes personalizarlo si usas otro formato.

b) `jakarta.faces.INTERPRET_EMPTY_STRING_SUBMITTED_VALUES_AS_NULL`

```
<context-param>
  <param-name>jakarta.faces.INTERPRET_EMPTY_STRING_SUBMITTED_VALUES_AS_NULL</param-name>
  <param-value>true</param-value>
</context-param>
```

- **Función:** Trata cadenas vacías como valores `null` en formularios. Esto puede simplificar la validación y lógica backend.
- **Valor recomendado:** `true`, especialmente si trabajas con formularios.

c) `jakarta.faces.CLIENT_WINDOW_MODE`

```
<context-param>
  <param-name>jakarta.faces.CLIENT_WINDOW_MODE</param-name>
  <param-value>url</param-value>
</context-param>
```

- **Función:** Controla cómo JSF maneja el estado del cliente. Usar `url` agrega un parámetro a las URLs, lo que puede mejorar la navegación entre ventanas.
- **Valor recomendado:** Según tus necesidades; `url` es útil si trabajas con navegación en múltiples ventanas.

d) `jakarta.faces.SERIALIZE_SERVER_STATE`

```
<context-param>
  <param-name>jakarta.faces.SERIALIZE_SERVER_STATE</param-name>
  <param-value>>false</param-value>
</context-param>
```

- **Función:** Si está habilitado, serializa el estado del servidor. Esto es útil en configuraciones de alta disponibilidad.
- **Valor recomendado:** `false` en la mayoría de los casos, salvo que tengas un clúster.

3. Otros parámetros útiles en aplicaciones grandes:

Manejo de Cache:

```
<context-param>
  <param-name>jakarta.faces.CACHE_MAX_SIZE</param-name>
  <param-value>100</param-value>
</context-param>
```

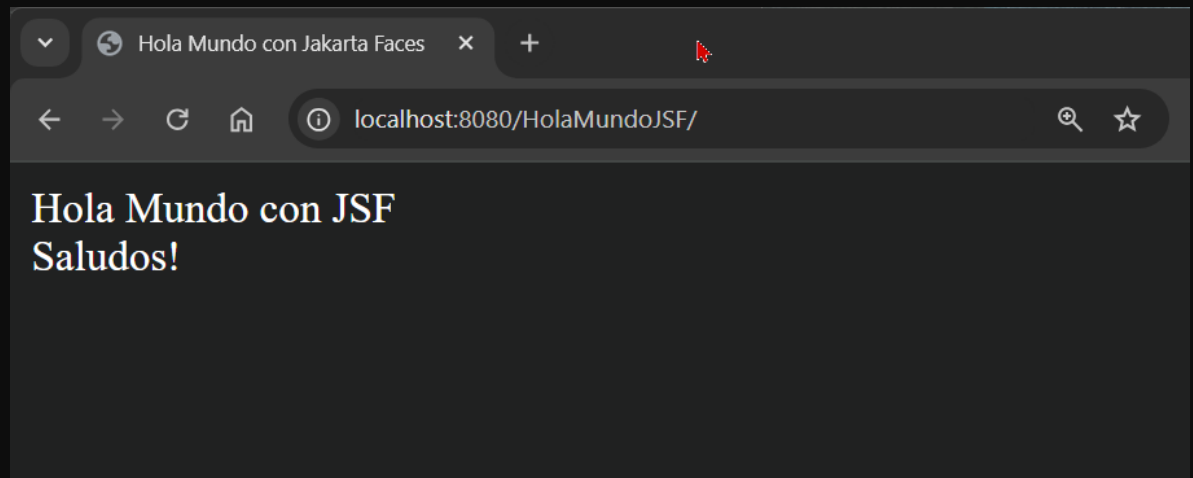
- **Función:** Define el tamaño máximo del caché para plantillas Facelets.

Estas son algunas configuraciones, pero recuerda que son opcionales, todo depende de las características de tu proyecto.

Despliegue y prueba

1. Inicia tu servidor GlassFish.
2. Despliega tu aplicación en Glassfish
3. Accede a tu aplicación desde el navegador en la URL

<http://localhost:8080/HolaMundoJSF/>



Conclusión

No hay necesidad de realizar ninguna otra configuración extra como en otras versiones, así que es todo lo que necesitas para crear tu aplicación de HolaMundo con Jakarta Faces.

En esta lección, has aprendido los conceptos básicos de Jakarta Faces para crear una aplicación simple de "Hola Mundo". En las próximas lecciones, profundizaremos en temas más avanzados y en el desarrollo de aplicaciones web completas utilizando Jakarta Faces.

¡Feliz codificación!

Saludos!

Ing. Ubaldo Acosta

Fundador de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)